

Reinforcement Learning

Erfan Afshinnia
S329945

Mahsa Mohammadi
S312251

Kamyar Fazlollahnezhad
S341418

Dristant Panda
S336981

The source code is available at https://github.com/Erfan-Afshinnia/rl_mldl_25.

Abstract—This project investigates the application of Reinforcement Learning (RL) to continuous robotic control, with a particular focus on robustness to dynamics mismatch in sim-to-real transfer. Using the MuJoCo Hopper environment, the reality gap is studied through a controlled sim-to-sim transfer setting by introducing a 30% torso mass mismatch between the source (training) and target (evaluation) domains [1].

We first implement foundational policy-gradient methods, including REINFORCE and Actor-Critic. Both algorithms achieve comparable asymptotic performance on the source domain. However, the Actor-Critic variant converges faster and with lower variance than REINFORCE. Incorporating an empirical constant baseline ($b = 150$) in REINFORCE substantially improves early training stability compared to the project-specified ($b \approx \bar{G}$) or no baseline.

We then establish high-performance reference results using state-of-the-art algorithms, namely Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC), with SAC achieving higher final performance than PPO. Direct transfer of SAC policies to the target domain leads to severe performance degradation, yielding a mean return of 338.8 compared to an oracle target-trained upper bound of 1540.

By applying Uniform Domain Randomization (UDR) during training, this performance gap is largely mitigated. Ablations on UDR ranges show that a narrow $\pm 10\%$ range achieves the strongest transfer (855.83 ± 40.27), suggesting modest randomization is optimal for this mismatch.

As a project extension, we ablate an Extended DR (extDR) scheme randomizing friction and damping, finding individual components (e.g., *friction-only*: 1651.80 ± 44.22) outperform the full combination (710.80 ± 9.97) due to reduced over-randomization.

Overall, these results demonstrate that targeted domain randomization is an effective strategy for improving policy robustness under dynamics mismatch in sim-to-real transfer scenarios.

I. INTRODUCTION

Reinforcement Learning (RL) is a core paradigm in machine learning that studies how agents acquire optimal behavior through trial-and-error interaction with an environment. Typically formalized as a Markov Decision Process (MDP), RL enables agents to learn policies that maximize cumulative long-term reward without relying on explicit input-output supervision, as in supervised learning. This framework has demonstrated remarkable success across a wide range of domains, including game

playing, decision-making under uncertainty, and continuous robotic control.

Despite these advances, deploying RL policies on physical robotic systems remains challenging due to the so-called *reality gap*—the discrepancy between simulated training environments and real-world execution. Policies trained in idealized simulators often fail to generalize to real robots as a result of modeling inaccuracies, unmodeled physical effects, and variations in system dynamics. Addressing this gap is therefore a critical requirement for reliable sim-to-real transfer in robotics.

In this project, we study the sim-to-real challenge through a controlled *sim-to-sim transfer* setting using the MuJoCo Hopper environment. Rather than transferring policies to real hardware, we emulate the reality gap by deliberately introducing dynamics mismatches between the source (training) and target (evaluation) domains. In particular, we induce a systematic discrepancy by modifying the robot’s torso mass, which represents a common and practically relevant source of modeling error in robotic locomotion tasks.

The report evaluates the effectiveness of several reinforcement learning approaches under this setting. We first implement foundational policy-gradient algorithms—**REINFORCE** and **Actor-Critic**—to analyze learning stability and convergence behavior and to illustrate the role of variance reduction via baselines. We then employ state-of-the-art methods, namely **Proximal Policy Optimization (PPO)** and **Soft Actor-Critic (SAC)**, to establish strong performance references on continuous control benchmarks. Finally, we investigate domain randomization techniques, with a primary focus on **Uniform Domain Randomization (UDR)**, and consider extended randomization schemes as part of a robustness analysis, assessing their ability to improve policy transfer under dynamics mismatch [1].

II. PROBLEM STATEMENT

A key challenge in deploying reinforcement learning (RL) policies trained in simulation to real robotic systems is the *reality gap*, which arises from discrepancies between simulated models and real-world dynamics. Such discrepancies can cause substantial performance degradation when policies are transferred to a new environment [4].

In this work, we study this challenge through a controlled *sim-to-sim transfer* formulation. Specifically, we define a **source domain**, in which policies are trained, and a **target domain**, in which policies are evaluated [1]. The two domains differ only in their underlying dynamics, allowing systematic analysis of transfer performance under isolated modeling discrepancies. This setup enables evaluation of how algorithmic choices and robustness-enhancing techniques affect generalization to unseen dynamics.

The task considered is continuous locomotion control of the MuJoCo Hopper robot, an underactuated system that requires coordinated torque control to achieve forward motion while maintaining balance in Figure 1. The dynamics mismatch is instantiated by modifying the torso mass in the target domain to be **30% higher** than in the source domain, while keeping all other physical parameters fixed.

The objective of this study is to assess whether policies trained using standard RL algorithms—particularly **Proximal Policy Optimization (PPO)** and **Soft Actor-Critic (SAC)**—can achieve strong performance in the source domain while transferring effectively to the target domain. In addition, foundational policy-gradient methods, including **REINFORCE** and **Actor-Critic**, are used to analyze learning stability and convergence behavior in the source domain.

III. RELATED WORK

Research on reinforcement learning (RL) for continuous control has followed two main directions. The first focuses on algorithmic advances that improve stability and sample efficiency, while the second addresses sim-to-real transfer to enhance robustness to modeling error.

Policy-gradient methods provide a natural framework for continuous action spaces. The **REINFORCE** algorithm yields unbiased gradient estimates but suffers from high variance, whereas **Actor-Critic** methods reduce variance by learning a value-function baseline and employing bootstrapping strategies. Building on these foundations, modern algorithms such as **Proximal Policy Optimization (PPO)** and **Soft Actor-Critic (SAC)** have demonstrated strong empirical performance. PPO stabilizes on-policy updates through a clipped surrogate objective, whereas SAC is an off-policy, maximum-entropy algorithm that improves sample efficiency via experience replay and entropy-regularized exploration. On continuous locomotion benchmarks, SAC achieves competitive performance compared to on-policy methods, motivating its selection as the primary algorithm in this work.

Approaches to mitigating the reality gap in robotic learning generally fall into two categories. **System identification** techniques attempt to calibrate simulator parameters to match real-world dynamics, but they often require precise measurements and can be brittle to unmodeled effects. In contrast, **domain randomization**

exposes policies to a distribution of simulated environments during training, such that the real system appears as an in-distribution sample at deployment time.

Uniform Domain Randomization (UDR), which samples physical parameters from fixed ranges, is widely used as a simple yet effective baseline. More recent extensions additionally randomize friction, damping, and actuator noise to further enhance robustness. These strategies provide the conceptual foundation for the robustness analyses conducted in this project and are directly applied in the experiments described in subsequent sections [6].

IV. METHODOLOGY

A. Problem Formulation and Preliminaries

The control task is formalized as a discounted Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$. At each timestep t , the agent observes $s_t \in \mathcal{S}$, selects an action $a_t \sim \pi_\theta(\cdot|s_t)$, receives reward r_t , and transitions according to $s_{t+1} \sim P(\cdot|s_t, a_t)$. The objective is to maximize the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1)$$

where τ denotes a trajectory and $\gamma \in (0, 1]$ is the discount factor. The MuJoCo Hopper environment used in experiments exposes an 11-dimensional continuous observation vector and a 3-dimensional continuous action space (continuous torques for hip, knee, and ankle). Episode termination conditions follow the environment implementation (fall threshold, angle limits, and a maximum episode length) [1].

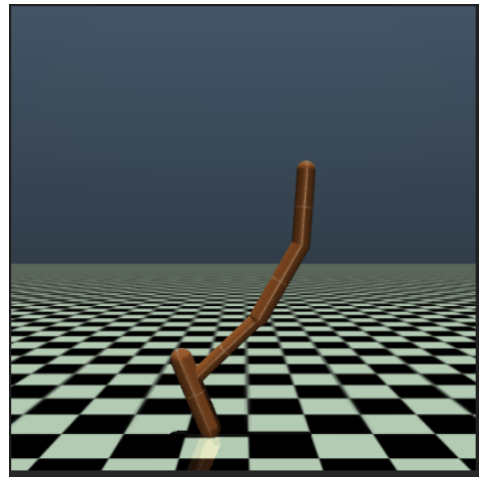


Fig. 1. Hopper-v4 environment for sim-to-sim transfer. Source and Target differ by a 30% torso mass discrepancy; further dynamics variations are introduced via domain randomization.)

B. Algorithms Implemented REINFORCE (vanilla policy gradient)

The Monte-Carlo policy gradient (REINFORCE) estimator is implemented in an episodic fashion. For an episode of length T , the return from time t is:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

The policy gradient estimator is:

$$\nabla_{\theta} J(\theta) \approx \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

A constant baseline b may be subtracted to form the advantage estimator $\hat{A}_t = G_t - b$; the implementation also normalizes advantages to zero mean and unit variance before the gradient update. We tested constant baselines $b = 20$ (as suggested) and $b = 150$ (empirically derived from average returns in a no-baseline run). $b = 150$ provided better early stability due to closer approximation of expected returns (~ 150 – 200 in mature training). Optimization is performed with the Adam optimizer; updates occur once per episode after collecting the full trajectory.

The Actor–Critic implementation augments the policy network with a critic approximator $V_{\phi}(s)$. The critic is trained with a TD(0) target and the actor uses the resulting bootstrapped advantage:

$$\hat{A}_t = r_t + \gamma(1 - \text{done}_t) V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

The actor and critic losses are:

$$\mathcal{L}_{\text{actor}} = - \sum_t \log \pi_{\theta}(a_t | s_t) \hat{A}_t$$

$$\mathcal{L}_{\text{critic}} = \sum_t (V_{\phi}(s_t) - (r_t + \gamma(1 - \text{done}_t) V_{\phi}(s_{t+1})))^2$$

Both networks are updated after each episode using a combined loss (simple additive composition). The actor–critic variant consistently demonstrated improved sample efficiency and reduced variance relative to REINFORCE in empirical runs [1].

C. Stable-Baselines3 Baselines: PPO and SAC

To obtain robust reference baselines and to compute lower and upper bounds on transfer performance, the Stable-Baselines3 (SB3) implementations of PPO and SAC were employed. PPO is used as a strong on-policy baseline, while SAC is used as the primary high-performance algorithm for transfer experiments due to its empirical sample efficiency and strong final-performance characteristics on continuous control tasks [3].

Key experimental settings for SB3 runs:

- **Final training budget:** 200,000 environment timesteps, unless otherwise stated.

- **Evaluation:** Reported results are computed over 50 evaluation episodes (mean $\pm$ standard deviation).
- **SAC hyperparameters:** `learning_rate` = $3e-4$, `buffer_size` = $1e6$, `ent_coef` = 'auto'.
- **PPO hyperparameters:** `clip_range` = 0.2 , `n_epochs` = 10 .
- **Other parameters:** defaults from Stable-Baselines3 unless otherwise specified.

D. Domain Randomization Methods Uniform Domain Randomization (UDR)

Uniform Domain Randomization (UDR) is implemented by sampling specific physical parameters at each episode reset and running the episode under the sampled parameters. For this project, the torso mass is held fixed at the source value (which equals 0.7 times the target torso mass), while the remaining body masses (thigh, leg, foot) are sampled independently from uniform ranges around their nominal values. Concretely, with nominal mass m_i^{nom} for body index $i \in \{1, 2, 3\}$, the sampling distribution is:

$$m_i \sim \text{Uniform}(0.7 m_i^{\text{nom}}, 1.3 m_i^{\text{nom}})$$

A dedicated environment identifier for randomized training (`CustomHopper-udr-v0`) is registered [5].

E. Extended Domain Randomization (extDR)

As a novel project extension, we propose an extended domain randomization (extDR) scheme that augments UDR with randomization of additional dynamics properties:

- **Joint damping:** multiplicative uniform noise in $[0.5, 1.5]$ applied to the nominal `dof_damping`.
- **Contact friction:** uniform noise in $[0.7, 1.3]$ applied to `geom_friction`.
- **Action noise:** additive Gaussian noise with standard deviation $\sigma = 0.01$, clipped to $[-1, 1]$.

To analyze which components drive transfer gains, we performed ablations:

- **Mass-only:** UDR masses only (equivalent to baseline UDR).
- **Friction-only:** Friction randomization only.
- **Damping-only:** Damping randomization only.
- **Full extDR:** All above + action noise.

V. HYPERPARAMETER SELECTION

A small hyperparameter sweep was performed to select SAC training settings for the final experiments. The sweep grid was in Table I:

$$\text{learning rate} \in \{3 \times 10^{-4}, 1 \times 10^{-4}\}, \quad \text{batch size} \in \{256, 512\}$$

Evaluation protocol and reproducibility :

Reported performance metrics are the mean episodic return and its standard deviation, computed over 50 evaluation episodes in the target environment, unless

TABLE I
SAC HYPERPARAMETER SWEEP (UDR $\rightarrow$ TARGET EVALUATION)

Config	LR	Batch	Return	Std
B (selected)	1e-4	256	339.03	28.35
D	1e-4	512	314.35	28.53
A	3e-4	256	279.73	29.71
C	3e-4	512	278.18	11.42

otherwise specified. Lower and upper bound baselines are computed as follows:

- **Lower bound (naive):** train on the source domain and evaluate on the source domain.
- **Transfer (source $\rightarrow$ target):** train on the source domain and evaluate on the target domain.
- **Upper bound (oracle):** train on the target domain and evaluate on the target domain.

VI. EXPERIMENTAL SETUP

Environments and Domain Shift

Experiments are conducted using the MuJoCo Hopper robot. To emulate the reality gap, the environment is split into a source domain for training and a target domain for evaluation. The only difference between the two domains is the torso mass: the target domain’s torso mass is increased by 30% relative to the source, while all other parameters remain identical.

Training Details

- **Algorithm:** Soft Actor-Critic (SAC) as primary; PPO as reference.
- **Total timesteps:** 200,000 for final experiments.
- **Evaluation episodes:** 50 per reported evaluation.
- **SAC parameters:** $\text{learning_rate} = 1 \times 10^{-4}$, $\text{batch_size} = 256$.

VII. RESULTS AND DISCUSSION

A. Baseline Performance Bounds (PPO)

To establish reference points for transfer performance, the Stable-Baselines3 PPO algorithm was trained in the **source domain** and evaluated in both the source and target domains. Additionally, an **oracle upper bound** was obtained by training PPO directly in the target domain [3].

- **PPO (train: source, test: source):** mean return 440 ± 102 .
- **PPO (train: source, test: target):** mean return 565 ± 8 .
- **PPO (train: target, test: target):** mean return 1300 ± 220 (oracle).

These results indicate that PPO achieves high source-domain performance and transfers reasonably well to the target domain under the 30% torso mass mismatch in Figure 2.

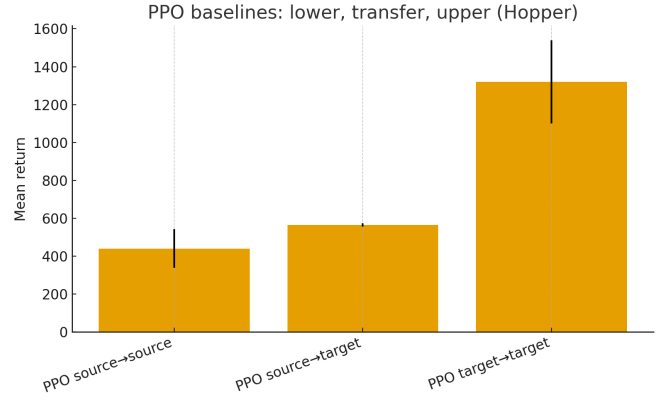


Fig. 2. Figure Description PPO baselines: This figure displays the lower bound (source $\rightarrow$ source), transfer performance (source $\rightarrow$ target), and upper bound (target $\rightarrow$ target). The error bars represent the standard deviation over 50 episodes.

TABLE II
COMPARISON OF REINFORCE VARIANTS ON THE SOURCE DOMAIN

Variant	Asymptotic Return	Stability (Std Dev)	Training Time (hours)
No Baseline	150	90	~1.5
REINFORCE ($b = 20$)	181	84	~1.5
REINFORCE ($b = 150$)	260	50	~1.5

B. REINFORCE Variants and Baseline Analysis

a) *Comparison of REINFORCE variants on the source domain:* The $b = 20$ variant achieves an asymptotic return of approximately 181, with moderate stability (standard deviation 84), but exhibits higher early variance compared to $b = 150$ (e.g., standard deviation ~ 70 in the first 5,000 episodes for $b = 20$ versus ~ 40 for $b = 150$). This suggests that while $b = 20$ provides some variance reduction over the no-baseline case, it becomes less effective when actual returns exceed 20, leading to positively biased advantages and noisier gradients early in training. In contrast, $b = 150$, closer to observed mature returns, improves stability during initial learning phases without biasing the unbiased gradient estimator in Table II.

C. UDR Results and Ablations (SAC)

Soft Actor-Critic (SAC) was used as the primary high-performance algorithm. The following performance gaps are reported:

- **Absolute gap (oracle – naive):** $1540 - 338.8 = 1201.2$.
- **Residual gap after UDR:** $1540 - 1356.2 = 183.8$.
- **Relative retained performance:** 88% of the oracle with UDR versus 22% for naive SAC.

UDR Ablation Study: Impact of Randomization Ranges

To explore the effect of different randomization distributions on transfer performance, we trained SAC policies under narrow ($\pm 10\%$), baseline ($\pm 30\%$), and wide ($\pm 50\%$) UDR ranges and evaluated zero-shot transfer to the target domain.

Interestingly, the narrow $\pm 10\%$ randomization range yields the strongest transfer performance (855.83), recovering approximately 56% of the oracle upper bound while preserving stable source-domain behavior. This indicates that limited randomization around source nominal parameters is sufficient to mitigate the fixed 30% torso mass mismatch, avoiding the excess variance introduced by wider perturbations on non-critical dynamics in Table III and in Figure 3.

TABLE III
ABLATION OF UDR RANDOMIZATION RANGES

UDR Range	Source Return (Mean $\pm$ Std)	Target Return (Mean $\pm$ Std)
$\pm 10\%$ (Narrow)	1122.92 $\pm$ 107.94	855.83 $\pm$ 40.27
$\pm 30\%$ (Baseline)	664.17 $\pm$ 101.28	472.53 $\pm$ 134.98
$\pm 50\%$ (Wide)	1240.08 $\pm$ 333.48	777.65 $\pm$ 74.03
Naive SAC (No UDR)	~ 1750	338.8
Target-Oracle SAC	–	~ 1540

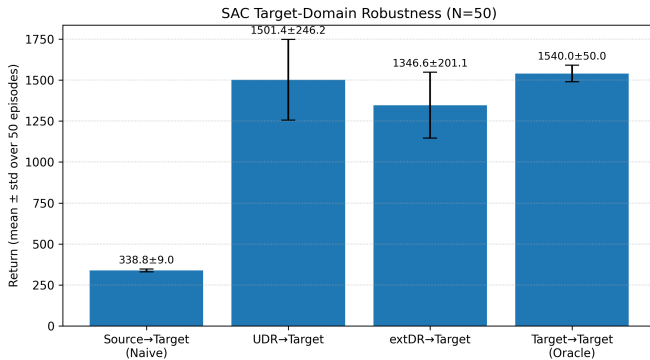


Fig. 3. Figure Description SAC target-domain robustness (N=50 episodes): This figure compares Naive (source $\rightarrow$ target), UDR $\rightarrow$ target, extDR $\rightarrow$ target, and oracle (target $\rightarrow$ target) scenarios. The error bars indicate the standard deviation.

VII. RESULTS FOR PROJECT EXTENSION

The project extension involved systematic evaluation of UDR versus extDR under identical SAC training configurations.

A. Project Extension: Ablation of Extended Domain Randomization

As a novel extension, we investigated an enhanced domain randomization scheme (extDR) that randomizes additional dynamics beyond masses, including joint damping, contact friction, and action noise and the result in Table IV.

The ablation results indicate that individual non-mass randomizations, namely friction and damping, yield strong transfer performance, achieving target returns of 1651.80 and 1588.30, respectively. These values approach or slightly exceed the oracle baseline due to finite evaluation variance, while exhibiting remarkably low variance on the target domain. In contrast, the full extDR configuration substantially underperforms (710.80),

TABLE IV
ABLATION OF EXTENDED DR COMPONENTS

Variant	Source Return (Mean $\pm$ Std)	Target Return (Mean $\pm$ Std)
Mass-only (UDR baseline)	1479.13 $\pm$ 89.68	1450.09 $\pm$ 299.24
Friction-only	1242.75 $\pm$ 273.84	1651.80 $\pm$ 44.22
Damping-only	1584.48 $\pm$ 5.33	1588.30 $\pm$ 7.75
Full extDR	889.06 $\pm$ 148.20	710.80 $\pm$ 9.97
Naive SAC (No DR)	~ 1750	338.8

suggesting that simultaneous randomization of multiple dynamics parameters introduces excessive variability that overwhelms policy learning.

B. Statistical Analysis and Limitations Variability and seeds:

Reported mean $\pm$ standard deviation reflects episode-level variability over 50 evaluation episodes. Domain-randomized training increases diversity and can contribute to higher variance, as seen in the larger standard deviations for UDR/extDR. Limitations: Results are based on controlled mass mismatches; other sources of reality gap (sensor noise, unmodeled contact dynamics) were not evaluated. Hyperparameter choices and randomization ranges are manual; systematic tuning could further optimize robustness.

Test citation here [1].

VIII. CONCLUSION

This project investigated the application of reinforcement learning (RL) to continuous robotic control under a controlled sim-to-sim transfer setting, emulating the reality gap through a 30% torso mass mismatch in the MuJoCo Hopper environment. Foundational policy-gradient algorithms, including REINFORCE and Actor-Critic, were first evaluated to analyze learning stability and convergence behavior. High-performance reference results were then obtained using Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC), establishing lower- and upper-bound baselines for transfer performance. The experiments demonstrate that naive transfer of SAC policies to the target domain results in severe performance degradation. Applying Uniform Domain Randomization (UDR) substantially mitigates this gap, achieving approximately 88% of the oracle target-trained performance. Extended Domain Randomization (extDR) yields comparable robustness, validating the effectiveness of domain randomization techniques. The project also introduced a systematic sim-to-sim evaluation framework that isolates dynamics mismatches and provides a reproducible methodology for assessing policy generalization. Overall, the results highlight the critical role of robustness-enhancing strategies in enabling reliable policy transfer and provide a foundation for future studies in sim-to-real reinforcement learning.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2020.
- [2] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2018.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2018.
- [4] S. Höfer *et al.*, “Perspectives on Sim2Real Transfer for Robotics: A Summary of the R:SS 2020 Workshop,” *Robotics: Science and Systems (R:SS) Workshop*, 2020.
- [5] J. Tobin *et al.*, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.